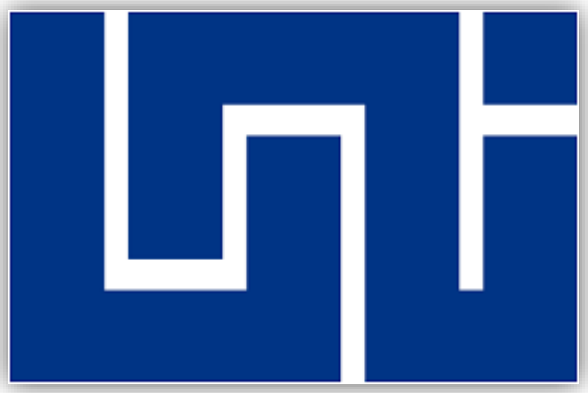
UNIVERSIDAD NACIONAL DE INGENIERÍA

ÁREA DEL CONOCIMIENTO DE LA INFORMACIÓN Y COMUNICACIÓN



ASIGNATURA:

PROGRAMACIÓN DE ESCRITORIO

PROGAMA ACADÉMICO:

INGENIERÍA EN SISTEMAS

DOCUMENTO TÉCNICO | MANUAL DE USUARIO DEL PROYECTO CODEPLAY: REGISTRO DE VENTAS.

INTEGRANTES:

- MAYKEL IVÁN SOTELO MARTÍNEZ (2025-0265U)
- HACNER JOSUÉ CUADRA ALVARADO (2025-0205U)
- JAVIER ALEXANDER LÓPEZ ANDINO (2025-0275U)
- JESHUA ALEXANDER GUADAMÚZ BOX (2025-2143U)

DOCENTE:

- ING. WALTER JOSÉ QUINTERO ACEVEDO

PERÍODO ACADÉMICO: II SEMESTRE 2026

MANAGUA, 31 DE OCTUBRE DEL 2026

Contenido

I. Resumen Ejecutivo....6
II. Problema y justificación....7
III. Objetivos....8
IV. 1.1.1. Implementar la lógica de negocio mediante principios de Programación Orientada a Objetos, gestionando el control de flujo, eventos de usuario y las reglas para el registro de ventas. 8
V. 1.1.2. Establecer mecanismos de validación de entradas y control de excepciones para garantizar la integridad de los datos y la robustez del sistema frente a errores en tiempo de ejecución. 8
VI. 1.1.3. Integrar el almacenamiento y consulta de la información mediante estructuras de datos en memoria o conexión a base de datos para mantener la persistencia y trazabilidad de las operaciones. 8
VII. Alcance y limitaciones 9
VIII. Análisis de proyecto 10
IX. Requerimientos Funcionales 10
X. Requerimientos No Funcionales....11
XI. Arquitectura del sistema 12
XII. Diagrama de dependencia entre capas 12
XIII. Responsabilidades de capa....13
XIV. Patrón arquitectónico MVP (Model-View Presenter) 13
XV. Mecanismo de inyección de dependencias 14
XVI. Diagrama de despliegue....16
XVII. Cumplimiento del Principio de Inversión de Dependencias (DIP - SOLID)....17
XVIII. Modelado UML 19
XIX. Modelo de datos....20
XX. Programación concurrente 30
XXI. Generación de reportes....32
XXII. Manejos de errores y logger 33
XXIII. Pruebas realizadas....34
XXIV. Conclusiones técnicas....35
XXV. Referencias 36
XXVI. MANUAL DE USUARIO 39
XXVII. Introducción y público objetivo....39
XXVIII. Requisitos del sistema....40

XXIX. Procedimiento de instalación 41
XXX. Credenciales de acceso y roles 43
XXXI. Guía de navegación por pantallas 44
XXXII. Casos de uso 50
XXXIII. Manejo de errores comunes 55
XXXIV. Glosario 58
XXXV. FAQ´s 60

Resumen Ejecutivo

Codeplay es un sistema de registro y gestión que permite el manejo optimo y eficaz de una cantidad grande de información dentro de un programa condensado con las herramientas necesarias para poder crear, visualizar, editar y gestionar los usuarios y las ventas de un videojuego, usando como herramienta principal el ecosistema de Windows Forms para el diseño de una interfaz amigable e intuitiva, de esta forma, el usuario podrá llevar a cabo todas las tareas antes mencionadas.

Problema y justificación

En la industria de distribución de videojuegos y gestión de comunidades en línea, el crecimiento acelerado de usuarios activos y transacciones comerciales suele colapsar los procesos administrativos cuando estos se manejan mediante herramientas manuales o fragmentadas. En este contexto de negocio, Codeplay aborda la desconexión crítica entre el registro comercial de compras y ventas del videojuego y la moderación activa de su comunidad de jugadores. Actualmente, actores clave como los administradores, moderadores y operadores de soporte enfrentan puntos de dolor severos: la información demográfica e historial de los usuarios reside en silos aislados, el seguimiento de incidencias (*Usuarios Reportados*, *Usuarios Suspendidos* y *Reportes en Revisión*) carece de trazabilidad en tiempo real, y la consolidación manual de métricas retrasa la toma de decisiones gerenciales. Esta desarticulación operativa no solo propicia errores humanos y vulnerabilidades en la integridad de los datos financieros, sino que también enlentece la aplicación de sanciones disciplinarias (como reportar, eliminar o banear cuentas), afectando directamente la experiencia y retención de los jugadores finales.

Para resolver esta problemática de raíz, el sistema propone una plataforma centralizada bajo una arquitectura cliente-servidor en **.**NET y SQL Server, diseñada para unificar el ciclo de vida del usuario, el registro transaccional y la generación automatizada de reportes y estadísticas. La elección de una aplicación nativa de escritorio en Windows Forms frente a alternativas web o móviles se justifica por las exigencias de seguridad, rendimiento y ergonomía propias de una herramienta de administración interna (Back Office). A nivel de ciberseguridad, operar en un entorno de escritorio dentro de la red local reduce drásticamente la superficie de ataque, evitando exponer paneles con privilegios críticos de moderación y bases de datos financieras al tráfico público de Internet, además de eliminar costos recurrentes de alojamiento web y servidores intermedios. Asimismo, el procesamiento nativo ofrece latencia cero en la carga y filtrado de grandes volúmenes de registros en tablas multicolumna e integración directa con el sistema de archivos local para la exportación de reportes, brindando una precisión y agilidad mediante teclado y monitor de alta resolución que resultaría inviable e insegura en pantallas móviles o navegadores dependientes del ancho de banda.

Objetivos

Objetivo General

Diseñar interfaces gráficas intuitivas en Windows Forms para la captura, consulta y visualización eficiente de datos de usuarios y transacciones de venta.

Objetivos Específicos

1.1.1. Implementar la lógica de negocio mediante principios de Programación Orientada a Objetos, gestionando el control de flujo, eventos de usuario y las reglas para el registro de ventas.

1.1.2. Establecer mecanismos de validación de entradas y control de excepciones para garantizar la integridad de los datos y la robustez del sistema frente a errores en tiempo de ejecución.

1.1.3. Integrar el almacenamiento y consulta de la información mediante estructuras de datos en memoria o conexión a base de datos para mantener la persistencia y trazabilidad de las operaciones.

Alcance y limitaciones

Codeplay registra y gestiona tanto la cantidad de usuarios como las transacciones realizadas dentro del sistema, como lo es la adquisición del videojuego, lo cual no está implementado como tal dentro la aplicación, por ende, se entiende que los usuarios agregados ya sea por el administrador u operadores han previamente atravesado la fase de compra y del registro mediante un proceso automatizado, con el cual igualmente no se cuenta dentro de la aplicación. De igual forma, la aplicación no está enfocada al diseño a gran escala, en este sentido, únicamente tolera cantidades moderadas de datos e información, debido a su estructura y herramientas utilizadas durante su desarrollo. Del mismo modo, no cuenta con ninguna integración de servicios web o derivados, como API´s o conexión a servidores externos mediante protocolos de red, lo cual limita el uso de la aplicación de forma local. Referente a la opción de estadística implementada dentro del diseño de la aplicación, esta no es muy completa, muestra únicamente datos relevantes que pueden considerarse obvios, siendo que son datos presentes en otras tablas, solo que agrupados en una sola opción, limitando el uso de herramientas avanzadas para análisis de datos y creación de reportes más detallados.

Análisis de proyecto

Requerimientos Funcionales

Código	Descripción	Actor	Prioridad	Criterio De Aceptación
RF-01	Inicio de sesión seguro mediante conexión SQL, basado en roles (Admin, Operador, Lector)	Todos los usuarios	Alta	El sistema valida credenciales contra la base de datos en menos de 2 s y muestra únicamente los módulos autorizados según el rol asignado.
RF-02	Sistema de recuperación de credenciales mediante validación de pregunta secreta previamente establecida	Todos los usuarios	Alta	Si el criterio (pregunta secreta) no es validado, el programa no permitirá el cambio de credenciales, protegiendo la integridad del mismo
RF-03	Sistema de registro de usuarios, con validación de datos de entrada, mantiene la integridad de los datos ingresados	Administrador / Operadores	Alta	Validaciones de datos de entrada de acuerdo a los parámetros establecidos, tolerancia de errores y el correcto almacenamiento de los datos.
RF-04	Panel únicamente accesible para roles de Administrador, con herramientas	Administrador	Alta	Manejo avanzado de la información ya existente manteniendo orden e

	como generación de reportes, listados y registro de usuarios.			integridad al implementar las nuevas funcionalidades, aprovechando las nuevas herramientas en uso.

Requerimientos No Funcionales

Código	Categoría	Descripción	Prioridad	Criterio
RNF-01	Rendimiento	Tiempo de espera de consultas en la base de datos	Alta	Las consultas realizadas no deben exceder 2s
RNF-02	Seguridad	Cifrado de contraseña y datos sensibles	Alta	Las contraseñas se almacenan mediante el algoritmo de SHA-256 al igual que las respuestas para la recuperación de credenciales
RNF-03	Usabilidad	Navegación accesible mediante atajos de teclado (*keyboard shortcuts*).	Media	Las operaciones dentro del sistema pueden realizar de igual forma usando atajos de teclado.

Arquitectura del sistema

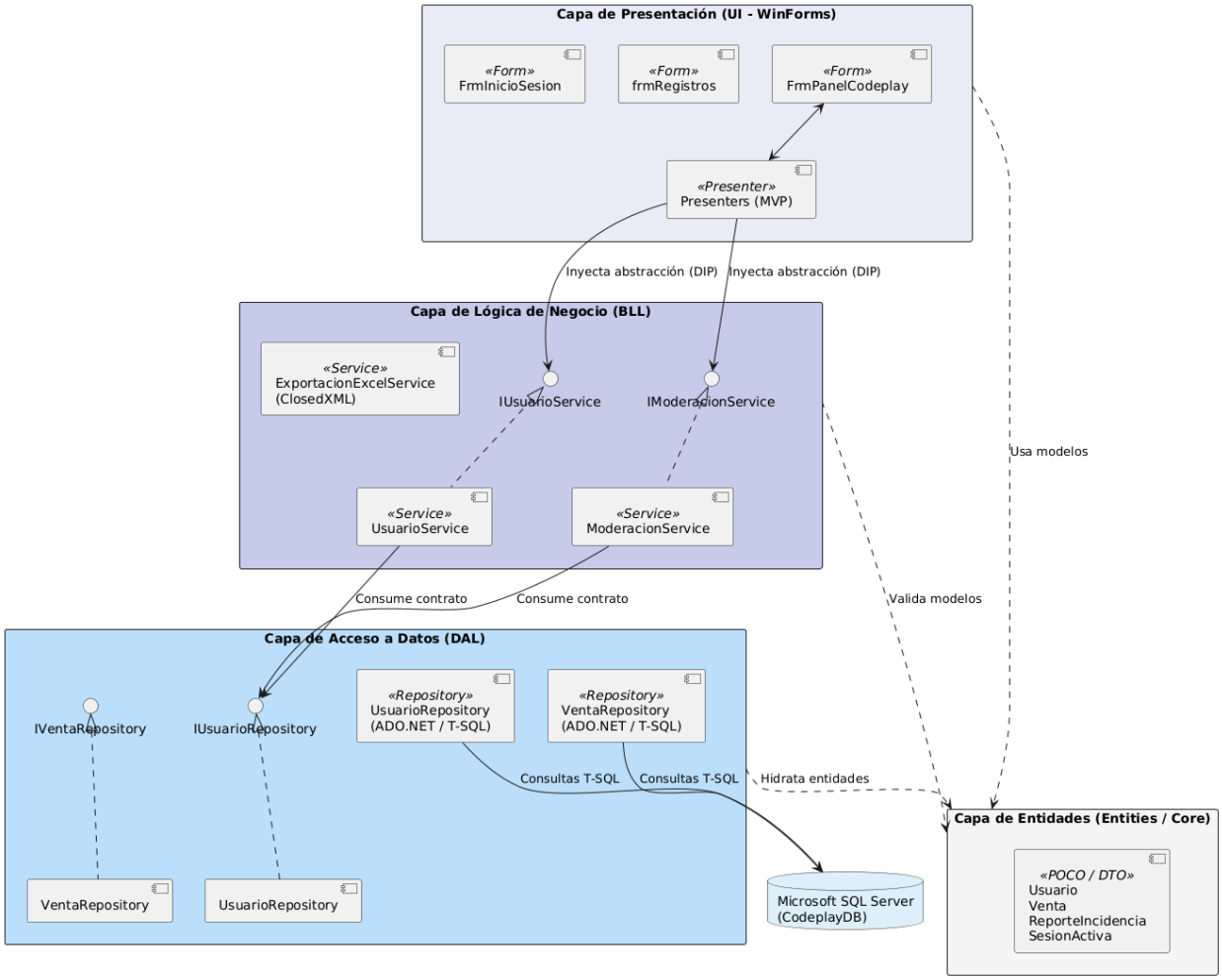


Diagrama de dependencia entre capas

El sistema Codeplay implementa una arquitectura multicapa estricta en .NET 8.0, separando las responsabilidades en cuatro capas lógicas: Capa de Presentación (UI), Capa de Lógica de Negocio (BLL), Capa de Acceso a Datos (DAL) y Capa de Entidades de Dominio / Contratos (Entities).

Responsabilidades de capa

- **UI (Presentación):** Compuesta por los formularios (FrmPanelCodeplay, frmRegistros, FrmInicioSesion) y controles visuales de ReaLTaiizor. Solo captura eventos del usuario (clics en btnListaBanear, btnListaReportar, mbtnGenerarReporte) y renderiza datos en controles como PoisonDataGridView, sin contener consultas SQL ni reglas financieras.
- **BLL (Lógica de Negocio):** Verifica reglas del dominio (ej. impedir que un usuario ya baneado realice una compra, validar mayoría de edad según FechaNacimiento, auditar cambios en EstadoCuenta, procesar métricas con LINQ y estructurar libros de Excel con ClosedXML).
- **DAL (Acceso a Datos):** Encapsula la comunicación con Microsoft SQL Server mediante cadenas de conexión parametrizadas y ejecución de procedimientos almacenados/consultas en T-SQL, retornando colecciones tipadas hacia la capa superior.
- **Entities (Entidades):** Contiene clases planas (POCO) como Usuario (IdUsuario, NombreUsuario, FechaNacimiento, CorreoElectronico, FechaRegistro, Pais, EstadoCuenta), SesionActiva y las interfaces contractuales que comunican a las demás capas sin generar referencias circulares.

Patrón arquitectónico MVP (Model-View Presenter)

- **Adecuación tecnológica a Windows Forms:** A diferencia de WPF o MAUI, Windows Forms no posee un motor nativo de enlace de datos bidireccional declarativo (Two-Way DataBinding basado en XAML y ICommand) que requiere MVVM. Forzar MVVM en WinForms exige escribir código repetitivo de notificación (INotifyPropertyChanged) que ensucia los controles visuales. Con MVP, el formulario (FrmPanelCodeplay) implementa una interfaz sencilla (IPanelCodeplayView), exponiendo propiedades limpias y delegando toda la orquestación al Presenter.
- **Desacoplamiento de librerías UI de terceros (ReaLTaiizor):** En FrmPanelCodeplay conviven componentes como PoisonDataGridView, MaterialButton y MaterialCard. Mediante MVP, la lógica de negocio ignora por completo si la tabla es un DataGridView estándar o un PoisonDataGridView: el Presentador únicamente entrega una colección IEnumerable<UsuarioDto> a la vista, facilitando cambios estéticos (como las transiciones de paletas de colores o reemplazo de botones) sin tocar una sola línea de reglas de negocio.
- **Control centralizado de moderación y estados de vista:** El panel administrativo conmuta dinámicamente entre vistas de Lista de Usuarios, Submenú de Reportes (Reportados, Suspendidos, En Revisión) y Estadísticas. El Presentador actúa como coordinador central de estas transiciones, respondiendo a eventos de moderación (Reportar, Eliminar, Banear) de forma determinista y facilitando las pruebas unitarias (Unit Testing) al poder simular (mockear) la vista sin levantar ventanas gráficas.

Mecanismo de inyección de dependencias

Para evitar el acoplamiento fuerte derivado de instanciar clases manualmente con new UsuarioService() dentro de los formularios, se implementó un contenedor de Inversión de Control (IoC) utilizando el paquete oficial Microsoft.Extensions.DependencyInjection en el punto de entrada de la aplicación (Program.cs).

```
using System;
using System.Windows.Forms;
using Microsoft.Extensions.DependencyInjection;
using Registro_de_ventas_Codeplay.BLL.Interfaces;
using Registro_de_ventas_Codeplay.BLL.Services;
using Registro_de_ventas_Codeplay.DAL.Interfaces;
using Registro_de_ventas_Codeplay.DAL.Repositories;

namespace Registro_de_ventas_Codeplay
{
   internal static class Program
   {
      public static IServiceProvider ServiceProvider { get; private set; }

      [STAThread]
      static void Main()
      {
         ApplicationConfiguration.Initialize();

         // 1. Crear la colección de servicios
         var services = new ServiceCollection();

         // 2. Configurar y registrar dependencias
         ConfigureServices(services);
```

```
            // 3. Construir el proveedor de servicios (Contenedor IoC)
            ServiceProvider = services.BuildServiceProvider();

            // 4. Resolver el formulario inicial inyectando sus dependencias automáticamente
            var mainForm = ServiceProvider.GetRequiredService<FrmPanelCodeplay>();
            Application.Run(mainForm);
        }

        private static void ConfigureServices(IServiceCollection services)
        {
            // --- Registro de Capa de Acceso a Datos (DAL) ---
            services.AddScoped<IUsuarioRepository, UsuarioRepository>();
            services.AddScoped<IVentaRepository, VentaRepository>();
            services.AddScoped<IReporteRepository, ReporteRepository>();

            // --- Registro de Capa de Lógica de Negocio (BLL) ---
            services.AddScoped<IUsuarioService, UsuarioService>();
            services.AddScoped<IModeracionService, ModeracionService>();
            services.AddSingleton<IExportacionExcelService, ExportacionExcelService>(); // ClosedXML

            // --- Registro de Formularios de la Capa de Presentación (UI) ---
            services.AddTransient<FrmInicioSesion>();
            services.AddTransient<FrmPanelCodeplay>();
            services.AddTransient<frmRegistros>();
        }
    }
}
```

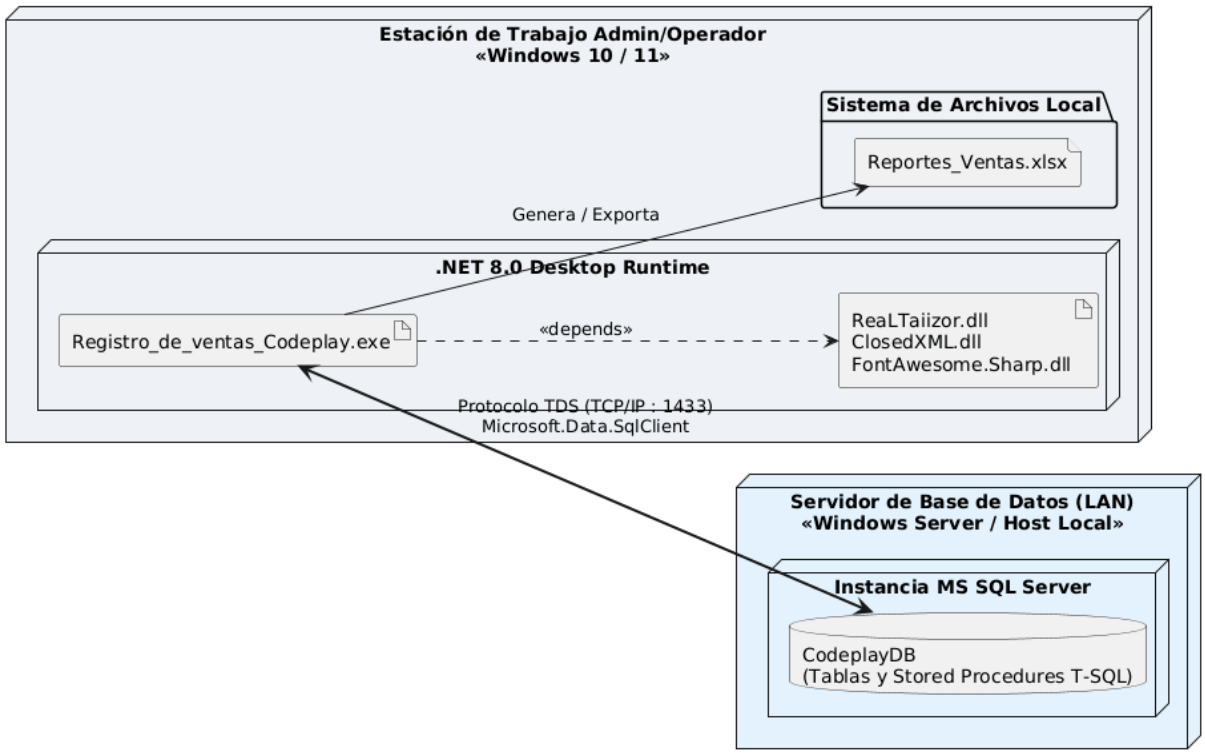
En los ciclos de vida utilizados tenemos:

AddScoped: Utilizado para repositorios (UsuarioRepository) y servicios de negocio (UsuarioService), asegurando una instancia limpia por cada operación/ámbito de trabajo y liberando conexiones de SQL Server oportunamente.

AddSingleton: Aplicado a servicios utilitarios sin estado mutable como ExportacionExcelService (generador de reportes con ClosedXML).

AddTransient: Aplicado a los formularios (FrmPanelCodeplay, frmRegistros), garantizando que cada vez que se abra una ventana modal o se reinicie sesión se genere una instancia fresca en memoria.

Diagrama de despliegue



El despliegue de Codeplay responde a un entorno corporativo de red local (LAN) bajo una topología Cliente-Servidor de dos nodos físicos, donde las estaciones de trabajo de los administradores y operadores se comunican mediante el protocolo tabular TDS (*Tabular Data Stream* sobre TCP/IP, puerto 1433) con el servidor central de base de datos, y exportan reportes directamente al sistema de archivos local.

Cumplimiento del Principio de Inversión de Dependencias (DIP - SOLID)

El Principio de Inversión de Dependencias (DIP) establece dos reglas fundamentales:

1. Los módulos de alto nivel no deben depender de los módulos de bajo nivel; ambos deben depender de abstracciones.
2. Las abstracciones no deben depender de los detalles, los detalles deben depender de las abstracciones.

En un diseño acoplado tradicional, FrmPanelCodeplay instanciaría directamente la clase concreta UsuarioService (y esta a su vez UsuarioRepository), creando una cadena rígida donde cualquier cambio en la forma de consultar SQL Server obligaría a recompilar y modificar el formulario.

En Codeplay, este flujo se invierte mediante Inyección por Constructor:

1. Definición de la Abstracción (Contrato): Se declara la interfaz IUsuarioService con las operaciones del dominio (ObtenerUsuarios(), BanearUsuario(int id), ReportarUsuario(int id)).
2. Inyección en el Módulo de Alto Nivel (FrmPanelCodeplay): El formulario administrativo desconoce la existencia de la clase UsuarioService y de SqlConnection. Únicamente solicita en su constructor una referencia a la interfaz IUsuarioService:

```
public partial class FrmPanelCodeplay : Form
{
   private readonly IUsuarioService _usuarioService;


   // El formulario recibe la abstracción por constructor (DIP)
   public FrmPanelCodeplay(IUsuarioService usuarioService)
   {
      InitializeComponent();
      _usuarioService = usuarioService;
   }


   private void CargarTablaUsuarios()
   {
      // Consume el contrato sin conocer los detalles de SQL Server
```

```
        dgvListaUsuarios.DataSource = _usuarioService.ObtenerUsuariosActivos();
    }

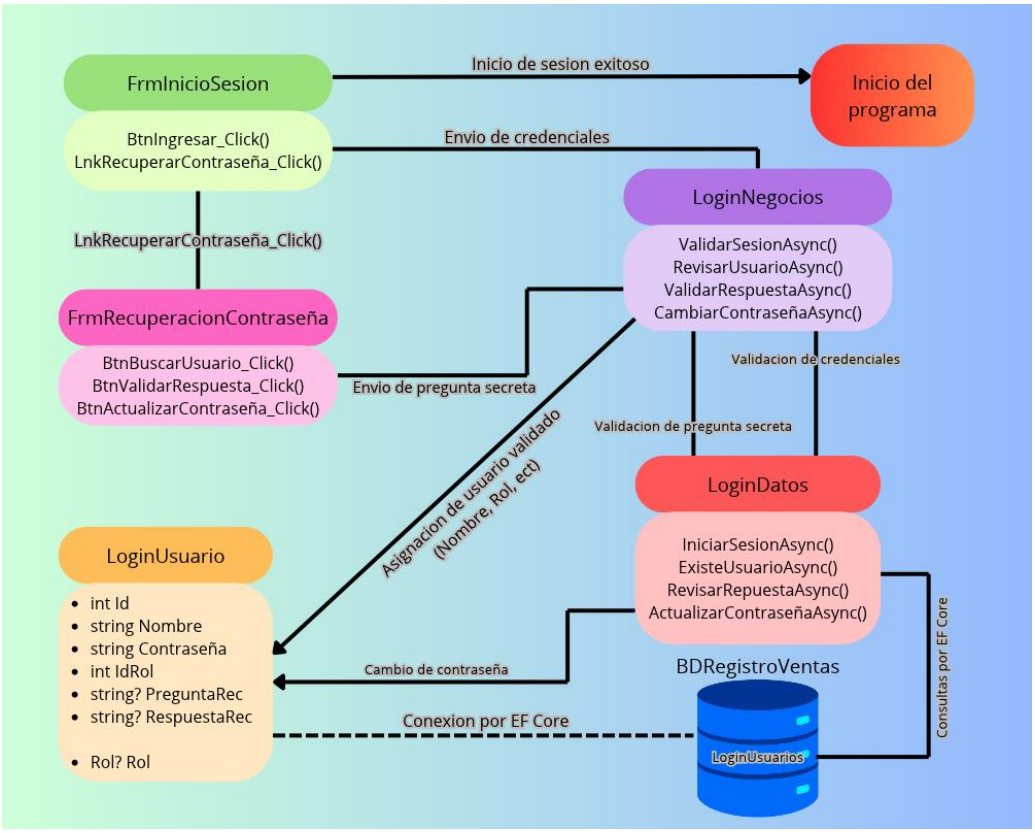
    private void btnListaBanear_Click(object sender, EventArgs e)
    {
        if (dgvListaUsuarios.CurrentRow != null)
        {
            int idUsuario = Convert.ToInt32(dgvListaUsuarios.CurrentRow.Cells["IdUsuario"].Value);
            _usuarioService.AplicarBaneo(idUsuario);
            CargarTablaUsuarios();
        }
    }
}
```

- **Implementación en el Módulo de Bajo Nivel (UsuarioService):** La clase concreta en la BLL implementa IUsuarioService y, aplicando exactamente el mismo principio hacia abajo, recibe en su constructor la abstracción IUsuarioRepository de la capa DAL. Gracias a esta inversión de dependencias, tanto la interfaz gráfica en FrmPanelCodeplay como las reglas de negocio en la BLL quedan protegidas ante cambios de infraestructura (por ejemplo, migrar de consultas ADO.NET puras a procedimientos almacenados o cambiar el motor de base de datos), permitiendo además realizar pruebas unitarias con datos simulados en memoria sin tocar la base de datos de producción.

Modelado UML

A continuación, se mostrará el diagrama sobre el sistema de gestiona y validación de inicio de sesión de usuarios, con sus relaciones con las capas DAL, BLL, UI y con la base de datos:

Figura 1.



Modelo de datos

El esquema abstrae las reglas esenciales del negocio para el registro de ventas, estructurando núcleos operativos interconectados. La entidad central Usuario sostiene la identidad del jugador en la plataforma y la entidad Login Usuarios para el control de credenciales, autenticación y preguntas de seguridad, mientras que la entidad Roles clasifica los permisos de acceso.

El modelo lógico traduce las abstracciones conceptuales a estructuras relacionales concretas optimizadas para Microsoft SQL Server. A continuación, se detalla el script que implementa las tablas, llaves primarias agrupadas, llaves foráneas con reglas explícitas de actualización y borrado, restricciones de validación de dominio e índices no agrupados estratégicos con columnas incluidas:

```
--
================================================================================
-- ESQUEMA LÓGICO DE BASE DE DATOS: BDRegistroVentas
--
================================================================================


USE [master];
GO


IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = N'BDRegistroVentas')
BEGIN
   CREATE DATABASE [BDRegistroVentas];
END
GO


USE [BDRegistroVentas];
GO


-- 1. TABLAS DE CATÁLOGO Y CONFIGURACIÓN
CREATE TABLE Roles (
```

```
    IdRol INT IDENTITY(1,1) NOT NULL,
    Nombre VARCHAR(50) NOT NULL,
    CONSTRAINT PK_Roles PRIMARY KEY CLUSTERED (IdRol),
    CONSTRAINT UQ_Roles_Nombre UNIQUE (Nombre)
);

CREATE TABLE Dispopais (
    Idpais INT IDENTITY(1,1) NOT NULL,
    NombrePais VARCHAR(30) NOT NULL,
    TipoMoneda VARCHAR(5) NOT NULL,
    Disponible BIT NOT NULL CONSTRAINT DF_Dispopais_Disponible DEFAULT (1),
    CONSTRAINT PK_Dispopais PRIMARY KEY CLUSTERED (Idpais),
    CONSTRAINT UQ_Dispopais_NombrePais UNIQUE (NombrePais)
);

-- 2. ENTIDADES DE GESTIÓN DE USUARIOS Y SEGURIDAD
CREATE TABLE Usuario (
    Idusuario INT IDENTITY(1,1) NOT NULL,
    NombreUsuario VARCHAR(30) NOT NULL,
    FechaNac DATE NOT NULL,
    CorreoElectronico VARCHAR(100) NOT NULL,
    Hashcontrasena VARCHAR(255) NOT NULL,
    FechaRegistro DATETIME NOT NULL CONSTRAINT DF_Usuario_FechaRegistro
DEFAULT (GETDATE()),
    Idpais INT NOT NULL,
    EstadoCuenta BIT NOT NULL CONSTRAINT DF_Usuario_EstadoCuenta DEFAULT
(1),
    RowVer ROWVERSION NOT NULL,
    CONSTRAINT PK_Usuario PRIMARY KEY CLUSTERED (Idusuario),
    CONSTRAINT UQ_Usuario_Correo UNIQUE (CorreoElectronico),
    CONSTRAINT UQ_Usuario_NombreUsuario UNIQUE (NombreUsuario),
```

```
    CONSTRAINT FK_Usuario_Dispopais FOREIGN KEY (Idpais)
        REFERENCES Dispopais (Idpais) ON DELETE NO ACTION
);

CREATE TABLE LoginUsuarios (
    IdUsuario INT IDENTITY(1,1) NOT NULL,
    NombreUsuario VARCHAR(50) NOT NULL,
    PasswordHash VARCHAR(64) NOT NULL,
    IdRol INT NULL,
    PreguntaRec VARCHAR(80) NOT NULL CONSTRAINT
DF_LoginUsuarios_PreguntaRec DEFAULT ('¿Cual es tu peli favorita?'),
    RespuestaRec VARCHAR(64) NOT NULL CONSTRAINT
DF_LoginUsuarios_RespuestaRec DEFAULT ('NO HASH!'),
    CONSTRAINT PK_LoginUsuarios PRIMARY KEY CLUSTERED (IdUsuario),
    CONSTRAINT UQ_LoginUsuarios_NombreUsuario UNIQUE (NombreUsuario),
    CONSTRAINT FK_LoginUsuarios_Roles FOREIGN KEY (IdRol)
        REFERENCES Roles (IdRol) ON DELETE SET NULL,
    CONSTRAINT CK_LoginUsuarios_PasswordHash CHECK (LEN(PasswordHash) =
64),
    CONSTRAINT CK_LoginUsuarios_RespuestaRec CHECK (LEN(RespuestaRec) >= 1)
);

CREATE TABLE Estadisticas (
    Idusuario INT NOT NULL,
    Horajugadas INT NOT NULL CONSTRAINT DF_Estadisticas_Horas DEFAULT (0),
    PartidasJugadas INT NOT NULL CONSTRAINT DF_Estadisticas_Partidas DEFAULT
(0),
    Victorias INT NOT NULL CONSTRAINT DF_Estadisticas_Victorias DEFAULT (0),
    Derrotas INT NOT NULL CONSTRAINT DF_Estadisticas_Derrotas DEFAULT (0),
    CONSTRAINT PK_Estadisticas PRIMARY KEY CLUSTERED (Idusuario),
    CONSTRAINT FK_Estadisticas_Usuario FOREIGN KEY (Idusuario)
```

```
        REFERENCES Usuario (Idusuario) ON DELETE CASCADE,
    CONSTRAINT CK_Estadisticas_Horas CHECK (Horajugadas >= 0),
    CONSTRAINT CK_Estadisticas_Partidas CHECK (PartidasJugadas >= 0),
    CONSTRAINT CK_Estadisticas_Victorias CHECK (Victorias >= 0),
    CONSTRAINT CK_Estadisticas_Derrotas CHECK (Derrotas >= 0)
);

-- 4. REGISTRO DE VENTAS, REEMBOLSOS Y AUDITORÍA
CREATE TABLE Venta (
    Idventa INT IDENTITY(1,1) NOT NULL,
    Idcopia INT NOT NULL,
    Idpais INT NOT NULL,
    Idusuario INT NOT NULL,
    Moneda VARCHAR(10) NOT NULL,
    PrecioTotal DECIMAL(10,2) NOT NULL,
    CONSTRAINT PK_Venta PRIMARY KEY CLUSTERED (Idventa),
    CONSTRAINT FK_Venta_CodCopiajuego FOREIGN KEY (Idcopia)
        REFERENCES CodCopiajuego (Idcopia) ON DELETE NO ACTION,
    CONSTRAINT FK_Venta_Dispopais FOREIGN KEY (Idpais)
        REFERENCES Dispopais (Idpais) ON DELETE NO ACTION,
    CONSTRAINT FK_Venta_Usuario FOREIGN KEY (Idusuario)
        REFERENCES Usuario (Idusuario) ON DELETE NO ACTION,
    CONSTRAINT CK_Venta_PrecioTotal CHECK (PrecioTotal >= 0.00)
);

CREATE TABLE TiendaVenta (
    Idventa INT IDENTITY(1,1) NOT NULL,
    Idusuario INT NOT NULL,
    Iditem INT NOT NULL,
```

```
    FechaCompra DATETIME NOT NULL CONSTRAINT DF_TiendaVenta_FechaCompra
DEFAULT (GETDATE()),
    MetodoCompra VARCHAR(15) NOT NULL,
    TotalPagado DECIMAL(10,2) NOT NULL,
    IdCupon INT NULL,
    CONSTRAINT PK_TiendaVenta PRIMARY KEY CLUSTERED (Idventa),
    CONSTRAINT FK_TiendaVenta_Usuario FOREIGN KEY (Idusuario)
      REFERENCES Usuario (Idusuario) ON DELETE NO ACTION,
    CONSTRAINT FK_TiendaVenta_Tienda FOREIGN KEY (Iditem)
      REFERENCES Tienda (Iditem) ON DELETE NO ACTION,
    CONSTRAINT FK_TiendaVenta_Cupones FOREIGN KEY (IdCupon)
      REFERENCES Cupones (Idcupon) ON DELETE SET NULL,
    CONSTRAINT CK_TiendaVenta_TotalPagado CHECK (TotalPagado >= 0.00)
  );

  CREATE TABLE CodCopiajuego (
    Idcopia INT IDENTITY(1,1) NOT NULL,
    Idusuario INT NOT NULL,
    FechaCompra DATETIME NOT NULL CONSTRAINT
DF_CodCopiajuego_FechaCompra DEFAULT (GETDATE()),
    NumCopia VARCHAR(100) NOT NULL,
    CONSTRAINT PK_CodCopiajuego PRIMARY KEY CLUSTERED (Idcopia),
    CONSTRAINT UQ_CodCopiajuego_NumCopia UNIQUE (NumCopia),
    CONSTRAINT FK_CodCopiajuego_Usuario FOREIGN KEY (Idusuario)
      REFERENCES Usuario (Idusuario) ON DELETE CASCADE
  );

  CREATE TABLE Auditoria (
    Idauditoria INT IDENTITY(1,1) NOT NULL,
    Idusuario INT NOT NULL,
    NombreTabla VARCHAR(50) NOT NULL,
```

```
    Idregistroafectado INT NOT NULL,
    Accion VARCHAR(10) NOT NULL,
    Valoranterior VARCHAR(MAX) NULL,
    Valornuevo VARCHAR(MAX) NULL,
    Fechamod DATETIME NOT NULL CONSTRAINT DF_Auditoria_Fechamod DEFAULT (GETDATE()),
    CONSTRAINT PK_Auditoria PRIMARY KEY CLUSTERED (Idauditoria),
    CONSTRAINT FK_Auditoria_Usuario FOREIGN KEY (Idusuario)
      REFERENCES Usuario (Idusuario) ON DELETE NO ACTION,
    CONSTRAINT CHK_Auditoria_Accion CHECK (Accion IN ('DELETE', 'UPDATE', 'INSERT'))
);

-- 5. ÍNDICES NO AGRUPADOS PARA OPTIMIZACIÓN
CREATE NONCLUSTERED INDEX IX_TiendaVenta_FechaCompra
ON TiendaVenta (FechaCompra ASC)
INCLUDE (Idusuario, Iditem, MetodoCompra, TotalPagado, IdCupon);

CREATE NONCLUSTERED INDEX IX_LoginUsuarios_NombreUsuario
ON LoginUsuarios (NombreUsuario)
INCLUDE (PasswordHash, IdRol, PreguntaRec, RespuestaRec);

CREATE NONCLUSTERED INDEX IX_Usuario_Idpais
ON Usuario (Idpais)
INCLUDE (NombreUsuario, CorreoElectronico, EstadoCuenta);
GO
```

La materialización física del esquema en la aplicación se implementa mediante Entity Framework Core 8 bajo el enfoque Code-First, permitiendo modelar las estructuras relacionales mediante clases tipadas en C# sin dependencias externas de diseño visual.

Las clases del modelo representan de forma pura las entidades de negocio. Se evitan los atributos de anotaciones de datos en sus propiedades para desacoplar el modelo de dominio de cualquier mecanismo específico de base de datos:

```
namespace Registro_de_ventas_Codeplay.Models
{
    public class Rol
    {
        public int IdRol { get; set; }
        public string? NombreRol { get; set; }
    }

    public class LoginUsuario
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public string Contraseña { get; set; } = string.Empty;
        public int IdRol { get; set; }
        public string? PreguntaRec { get; set; }
        public string? RespuestaRec { get; set; }

        // Para INNER JOIN en EF Core
        public Rol? Rol { get; set; }
    }
}
```

Todas las restricciones de esquema se centralizan en la clase LoginConexionEFCore sobrescribiendo el método OnModelCreating. Este enfoque proporciona control granular sobre los tipos de columnas de SQL Server, índices únicos, valores predeterminados y reglas de concurrencia optimista:

```
        public DbSet<LoginUsuario> LoginUsuarios { get; set; }
        public DbSet<Rol> Roles { get; set; }
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Rol>(entity =>
    {
        entity.ToTable("Roles");
        entity.HasKey(r => r.IdRol);
        entity.Property(r => r.IdRol).HasColumnName("IdRol");
        entity.Property(r => r.NombreRol).HasColumnName("Nombre").IsRequired();
    });

    modelBuilder.Entity<LoginUsuario>(entity =>
    {
        entity.ToTable("LoginUsuarios");

        // Definición de la Llave Primaria
        entity.HasKey(u => u.Id);

        // Mapeo de columnas
        entity.Property(u => u.Id)
              .HasColumnName("IdUsuario");

        entity.Property(u => u.Nombre)
              .HasColumnName("NombreUsuario")
              .HasMaxLength(50)
              .IsRequired();

        entity.Property(u => u.Contraseña)
              .HasColumnName("PasswordHash")
```

```
                .IsRequired();

            entity.Property(u => u.IdRol)
                .HasColumnName("IdRol")
                .IsRequired();

            entity.Property(u => u.PreguntaRec)
                .HasColumnName("PreguntaRec")
                .IsRequired();

            entity.HasOne(u => u.Rol)
                .WithMany()
                .HasForeignKey(u => u.IdRol);
        });
    }
```

El acceso a la base de datos satisface los requerimientos de diseño de las herramientas CLI de EF Core a través del estándar formal appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost\\SQLEXPRESS;Database=BDRegistroVentas;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

La creación y evolución de la base de datos se documenta a través de los comandos oficiales ejecutados en la Consola del Administrador de Paquetes:

```
PM> Add-Migration
Build started...
Build succeeded.
To undo this action, use 'Remove-Migration'.
```

```
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20261030041235_InitialCreate'.
Done.
```

La comprobación formal de la sincronización física del modelo se confirma inspeccionando la tabla del sistema __EFMigrationsHistory:

```
SELECT MigrationId, ProductVersion
FROM [BDRegistroVentas].[dbo].[__EFMigrationsHistory];
```

La elección de la estrategia Entity Framework Core Code-First obedece a la necesidad de mantener el modelo relacional sincronizado de forma determinista con el repositorio de Git del equipo de desarrollo. Al representar cada cambio estructural en clases C# con métodos simétricos Up() y Down(), se elimina el desfase manual de scripts y se habilita la capacidad de aplicar o revertir versiones del esquema sin alterar el historial transaccional de los ambientes de trabajo.

La consistencia e integridad referencial se garantiza a través de un esquema de validación dual. A nivel de infraestructura en SQL Server, las llaves foráneas delimitan el ciclo de vida de los datos, aplicando ON DELETE CASCADE en entidades para prevenir la existencia de registros huérfanos, y configurando ON DELETE NO ACTION o Restrict en tablas de registro para proteger la inmutabilidad contable del sistema. A nivel de aplicación, Entity Framework Core y la capa de negocio validan la integridad antes de emitir sentencias a la red mediante consultas asíncronas y la inspección continua del componente Change Tracker, interceptando inconsistencias y traduciéndolas a excepciones controladas en la interfaz de usuario.

Programación concurrente

La operación de larga duración en el sistema corresponde a la generación y renderizado de reportes en formato Excel mediante ClosedXML. Aunque la obtención de registros desde la base de datos es una operación de entrada/salida, la construcción del modelo de objetos del libro de cálculo, la aplicación de formatos condicionales, estilos tipográficos, bordes y la serialización del archivo comprimido .xlsx en memoria representan una carga intensiva de procesamiento. Para prevenir que los cálculos y la manipulación de celdas congelen el hilo principal de la interfaz, este procesamiento se descarga al grupo de subprocesos de .NET utilizando Task.Run

Cuando se ejecuta una llamada síncrona hacia SQL Server, el hilo de la interfaz se bloquea físicamente en espera de la respuesta de la red, dejando de atender los mensajes del sistema

operativo. Esto provoca que el sistema marque la ventana como "No responde", inhabilite el cursor y congele la interfaz.

El uso de async y await implementa el modelo asíncrono basado en tareas (Task-based Asynchronous Pattern o TAP). Cuando la ejecución alcanza la instrucción await, se registra una continuación y el hilo de la interfaz gráfica se libera de inmediato para retornar a su bucle de mensajes. En el momento en que SQL Server responde con los bytes de la consulta, la aplicación captura el resultado y reanuda la ejecución del método en el hilo de la UI exactamente donde se quedó.

Para permitir que el usuario interrumpa la generación de reportes o la consulta de base de datos antes de que finalicen, el sistema implementa el patrón de cancelación cooperativa a través de CancellationTokenSource. Este mecanismo se denomina cooperativo porque no destruye abruptamente el subproceso, sino que propaga una señal booleana que las diferentes capas consultan de forma voluntaria.

En la capa DAL, el token se transfiere directamente a los métodos nativos de ADO.NET, lo cual le permite al cliente enviar una señal de atención de red hacia SQL Server para abortar la consulta en el motor de base de datos. En la capa de negocio, el procesamiento de celdas de ClosedXML valida periódicamente la propiedad token.ThrowIfCancellationRequested() en bucles prolongados para detener la iteración y limpiar los recursos inmediatamente.

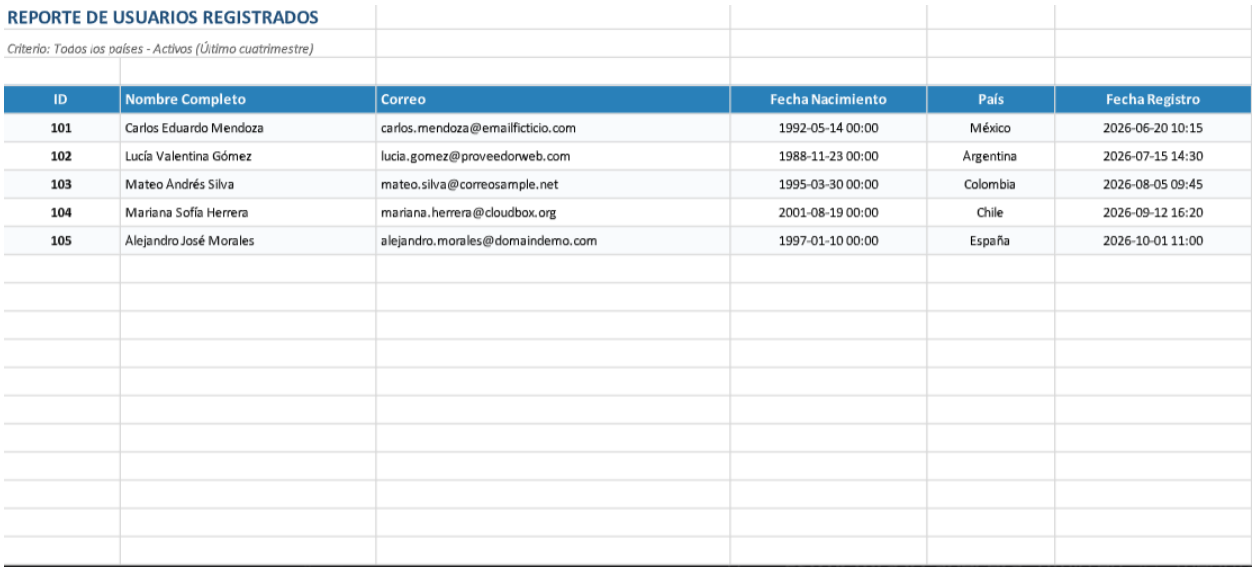
En la arquitectura implementada para este sistema, el acceso a datos y la lógica de negocio operan bajo un modelo sin estado mutable compartido entre hilos. Cada petición asíncrona hacia la base de datos gestionada por ADO.NET y Entity Framework Core instancia su propia conexión (SqlConnection) y su propio contexto (LoginDbContext) delimitados dentro de bloques using, garantizando que cada hilo trabaje con recursos de memoria y canales de comunicación aislados que son recolectados al finalizar la operación.

Al no existir colecciones globales en memoria, variables estáticas mutables ni archivos locales compartidos que sean leídos y escritos simultáneamente por múltiples tareas en segundo plano, no se presentan condiciones de carrera en el espacio de memoria de la aplicación. Por lo tanto, la aplicación deliberada de primitivas de sincronización como SemaphoreSlim resultaría en un sobrecoste innecesario de sincronización, ya que la consistencia transaccional y la concurrencia sobre los registros residen íntegramente en el motor de base de datos SQL Server mediante sus propios niveles de aislamiento de transacciones.

Generación de reportes

El paquete Nugget ClosedXMl maneja hojas de cálculo .xls para generar reportes con los datos existentes en la base de datos, permitiendo agrupar los datos referentes a las especificaciones correspondientes, la obtención se realiza mediante la consulta de la base de datos SQL utilizando ADO.NET por una fecha especificada por el usuario, los datos son consultados respetando los parámetros especificados y los exporta hacia un directorio que el usuario tendrá que elegir para guardarlo dentro de su máquina local.

Figura. Reporte generado usando el paquete Nugget.



REPORTE DE USUARIOS REGISTRADOS

Criterio: Todos los países - Activos (Último cuatrimestre)

ID	Nombre Completo	Correo	Fecha Nacimiento	País	Fecha Registro
101	Carlos Eduardo Mendoza	carlos.mendoza@emailficticio.com	1992-05-14 00:00	México	2026-06-20 10:15
102	Lucía Valentina Gómez	lucia.gomez@proveedorweb.com	1988-11-23 00:00	Argentina	2026-07-15 14:30
103	Mateo Andrés Silva	mateo.silva@correosample.net	1995-03-30 00:00	Colombia	2026-08-05 09:45
104	Mariana Sofía Herrera	mariana.herrera@cloudbox.org	2001-08-19 00:00	Chile	2026-09-12 16:20
105	Alejandro José Morales	alejandro.morales@domaindemo.com	1997-01-10 00:00	España	2026-10-01 11:00

Manejos de errores y logger

El programa maneja cuando el usuario ingresa datos invalido que no cumplen con los requerimientos especificados, maneja campos vacíos, restricción de datos duplicados. La capa BLL usa un sistema de tupla que contiene un valor booleano que verifica si fue un éxito la operación, un valor string que muestra información más detallada del error para ser consumida por la capa de presentación, y un valor opcional que depende del contexto. Logger no implementado

debido a que la ejecución de tareas del programa es bastante sencilla, así que por motivos de rendimiento y eficiencia no fue añadido a la versión final, sino hasta futuras versiones

Pruebas realizadas

Conclusiones técnicas

El desarrollo e implementación de Codeplay demuestra la efectividad de combinar una arquitectura multicapa desacoplada (UI, BLL, DAL y Entities) bajo el patrón MVP (*Passive View*) y el Principio de Inversión de Dependencias (DIP) en .NET 8.0. Mediante el uso de inyección de dependencias nativa (Microsoft.Extensions.DependencyInjection) y la persistencia relacional en Microsoft SQL Server con T-SQL, el sistema resuelve la fragmentación operativa entre el registro comercial de compras y ventas del videojuego y la moderación disciplinaria de su comunidad de

jugadores (*Reportar, Suspender, Banear y Eliminar*). Esta separación estricta de responsabilidades permitió evolucionar la interfaz gráfica, integrar componentes de código abierto de ReaLTaiizor (MaterialCard, MaterialButton, PoisonDataGridView) con una paleta cromática ergonómica en tonos azul-celeste pastel y gestionar el código fuente mediante Git y GitHub, todo ello sin comprometer la estabilidad del diseñador ni alterar las reglas de negocio subyacentes.

Asimismo, la elección de una solución nativa de escritorio en Windows Forms consolidó un entorno de administración interna (Backoffice) de alta seguridad y rendimiento superior frente a alternativas web o móviles. Al operar dentro de la red local con procesamiento en memoria mediante LINQ, control de acceso basado en roles (SesionActiva) y generación automatizada de reportes locales con ClosedXML, la plataforma garantiza latencia cero en la auditoría de grandes volúmenes de registros, integridad referencial en cada transacción y una superficie de ataque reducida, entregando una base de software robusta, mantenible y preparada para escalar.

Referencias

- ClosedXML. (2024). *ClosedXML documentation: The easy way to OpenXML*. Read the Docs. https://closedxml.readthedocs.io/en/latest/
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

- IgnaceMaes. (2021). *MaterialSkin: Google's Material Design skin for .NET WinForms (C#/VB.Net)* [Software de computador]. GitHub. https://github.com/IgnaceMaes/MaterialSkin
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Microsoft Corporation. (2024a). *Inyección de dependencias en .NET*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/core/extensions/dependency-injection
- Microsoft Corporation. (2024b). *Información general de Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/
- Microsoft Corporation. (2024c). *Información general sobre formularios Windows Forms (.NET)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/overview/
- Microsoft Corporation. (2024d). *Language Integrated Query (LINQ) en C#*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/linq/
- Microsoft Corporation. (2024e). *Documentación técnica de Microsoft SQL Server y Transact-SQL (T-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/sql-server/
- Taiizor. (2024). *ReaLTaiizor: .NET WinForms UI library (Material, Poison, Cyber & Crown themes)* [Software de computador]. GitHub. https://github.com/Taiizor/ReaLTaiizor

MANUAL DE USUARIO

Introducción y público objetivo

Codeplay es un sistema de gestión y registro de usuarios y ventas de un videojuego, permite llevar el registro de cantidades grandes de información, como lo son nombres de usuarios, correos, contraseñas en un solo sistema diseñado específicamente para este problema, va dirigido a todos aquellos usuarios que desean crear sistemas similares y que desean optimizar la gestión y almacenamiento de información a gran escala. Dentro de este sistema, cada persona tiene definido su rol del cual forma parte dentro del mismo, como lo son los administradores, quienes poseen el acceso total a las funcionalidades del sistema, los operadores que tienen acceso parcial al registro

de usuarios mas no a ciertas opciones avanzadas y finalmente los lectores, los cuales únicamente pueden visualizar la información que está el sistema pero no editarla o modificarla.

Requisitos del sistema

A continuación se piden los requisitos mínimos del sistema para que este pueda ejecutarse sin problemas:

(a) Sistema Operativo Windows 10 o superior (64 bits)
(b) .NET FRAMEWORK Desktop enviroment (versión 8 o superiores)
(c) SQL SERVER 2019 EXPRESS
(d) Visual Studio 2022 o superior
(e) Git para Windows

(f) Memoria RAM mínima (2GB)
(g) Al menos 5GB de espacio interno

Procedimiento de instalación

Paso 1. Descargar el proyecto desde GitHub

Opción A:

1. Abrir la terminal (Git Bash, CMD o PowerShell).
2. Clonar el repositorio oficial ejecutando:

   git clone https://github.com/Crossack/Registro_de_ventas_Codeplay.git

3. Ingresar al directorio del proyecto:

cd Registro_de_ventas_Codeplay

Opción B:

1. Ingresar el enlace https://github.com/Crossack/Registro_de_ventas_Codeplay del repositorio en GitHub desde el navegador web.
2. Hacer clic en el botón verde *Code* ubicado en la parte superior derecha.
3. Seleccionar la opción *Download ZIP*.
4. Extraer el contenido del archivo descargado en una ruta accesible y sin permisos restringidos (por ejemplo, C:\Users\Dell\Desktop).

Paso 2. Configuración de la base de datos

1. Iniciar SQL Server Management Studio (SSMS) y conectarse a la instancia del equipo (por ejemplo, localhost\SQLEXPRESS o con autenticación de Windows).
2. Abrir el archivo de base de datos ubicado en la carpeta del repositorio (BDRegistroVentas.sql).
3. Presionar F5 o hacer clic en Execute para crear la base de datos BDRegistroVentas, sus tablas e índices.
4. Verificar en el Explorador de Objetos que la base de datos aparezca listada y con sus tablas creadas (Usuario, LoginUsuarios, Roles, TiendaVenta, etc.).

Paso 3. Vinculación de la cadena de conexión

En la raíz del directorio de ejecución (bin\Debug\net8.0-windows), el sistema lee o genera el archivo conexion.txt. Si es la primera vez que se ejecuta, el sistema generara una cadena por defecto, aunque se recomienda modificarlo por la cadena proporcionada por SSMS. La cadena debe tener la siguiente estructura:

Server=localhost\SQLEXPRESS;Database=BDRegistroVentas;Trusted_Connection=True;TrustServerCertificate=True;

Paso 4. Compilación y primera ejecución

1. Abrir el archivo de solución (.sln) con Visual Studio 2022.
2. Hacer clic derecho sobre la solución en el Explorador de Soluciones y seleccionar Restaurar paquetes NuGet para descargar automáticamente las librerías requeridas (ClosedXML, Entity Framework Core y proveedores de SQL Server).
3. Compilar el proyecto presionando Ctrl + Shift + B y comprobar que no existan errores de compilación.
4. Presionar F5 o hacer clic en el botón Iniciar para lanzar la aplicación.
5. Iniciar sesión en el formulario principal utilizando el usuario administrativo de pruebas configurado en la base de datos.

Credenciales de acceso y roles

Las credenciales de acceso configuradas en el sistema fueron hechas para mostrar la lógica de negocios implementadas, en total, se realizaron 4 credenciales de acceso, 1 como administrador, 2 como operadores y 1 únicamente como lector. A continuación se muestra la tabla con los nombres de usuario y contraseñas (para uso demostrativo de las capacidades del sistema).

Se le recuerda al lector que estas credenciales son únicamente con el propósito de usarlas y verificar como funciona la lógica de negocio y como estos procesos se implementan dentro del sistema para hacerlo completamente funcional.

Administrador	Operador 1	Operador 2	Lector
Crossack	SoteloMaykel	Hacner	Alexbox07

@metMet0000	HolasoynoobXD1918!	Hacner2301	545347A#1e

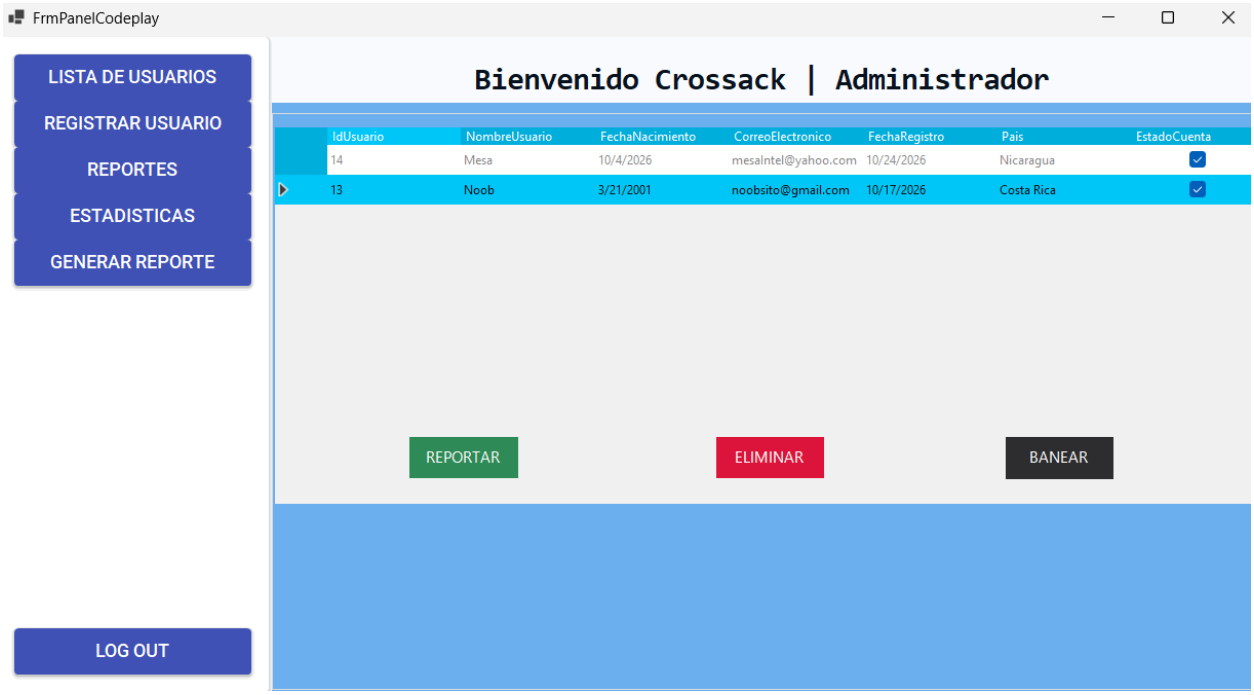
Guía de navegación por pantallas

Figura 1.



Primeramente, se mostrará la imagen de inicio de sesión en donde el usuario pondrá las credenciales correspondientes. Aquí encontraremos el campo de usuario y contraseña, una vez ingresado el usuario y la contraseña correspondiente, presione ingresar para poder acceder a la interfaz de Codeplay.

Figura 2.



Si es el administrador quien ingresa se le mostrará la siguiente ventana, que se divide en tres grandes partes, a la derecha el panel de todas las opciones que el administrador posee con altos privilegios, como listar los usuarios, generar reportes, mostrar estadísticas y generar reportes, al igual que reportar, eliminar y banear usuarios. Desde este panel, si se hace click en el botón de "Registrar Usuarios" se le enviará a la siguiente ventana:

Figura 3.



Panel de registro: Este panel está habilitado tanto para el rol de operador como para el rol de administrador, desde el mismo, se llevan a cabo el registro con todas las funciones C.R.U.D integradas.

Registrar: Registra los datos de los nuevos usuarios que sean agregados ya sea por el administrador u operadores.

Editar: Permite modificar la información de los usuarios que ya se encuentran en la base de datos, únicamente se selecciona un usuario del panel de usuarios y haz Click en “Editar” para comenzar con el proceso.

Eliminar: Borra de forma permanente el usuario con los datos pertenecientes a este mismo de la base de datos.

Salir: Si todas las acciones del panel han sido finalizadas entonces seleccione “Salir” para cerrar volver a la ventana de inicio se sesión.

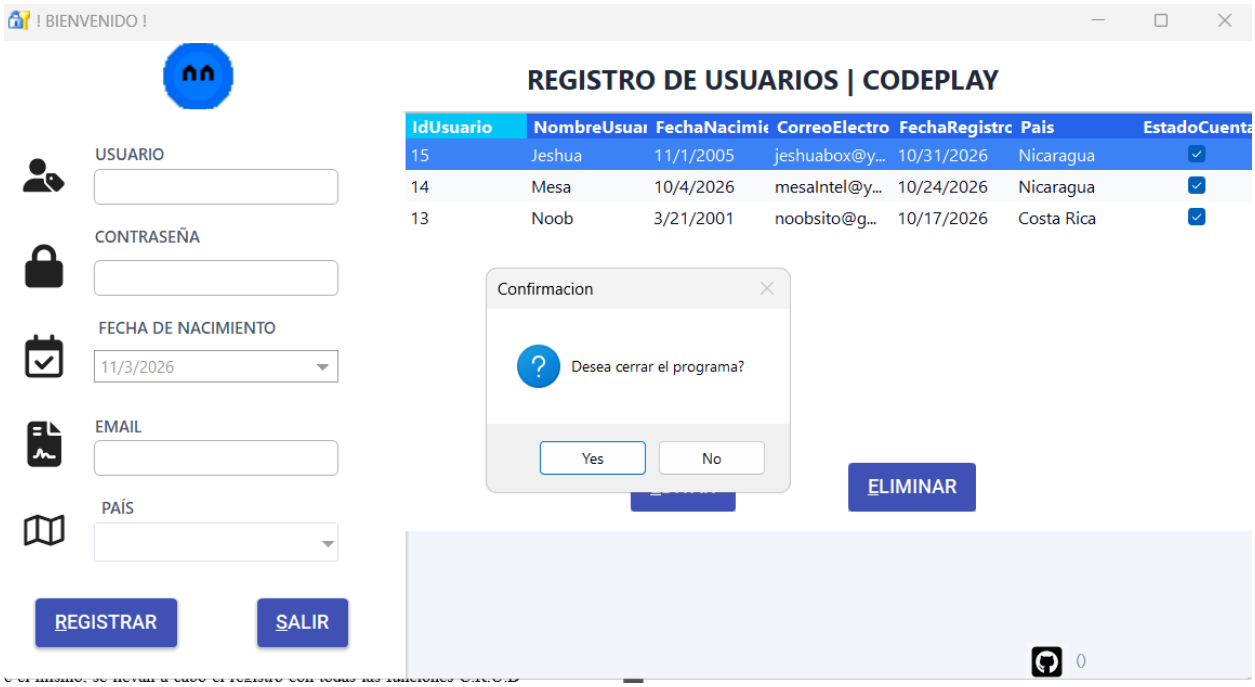
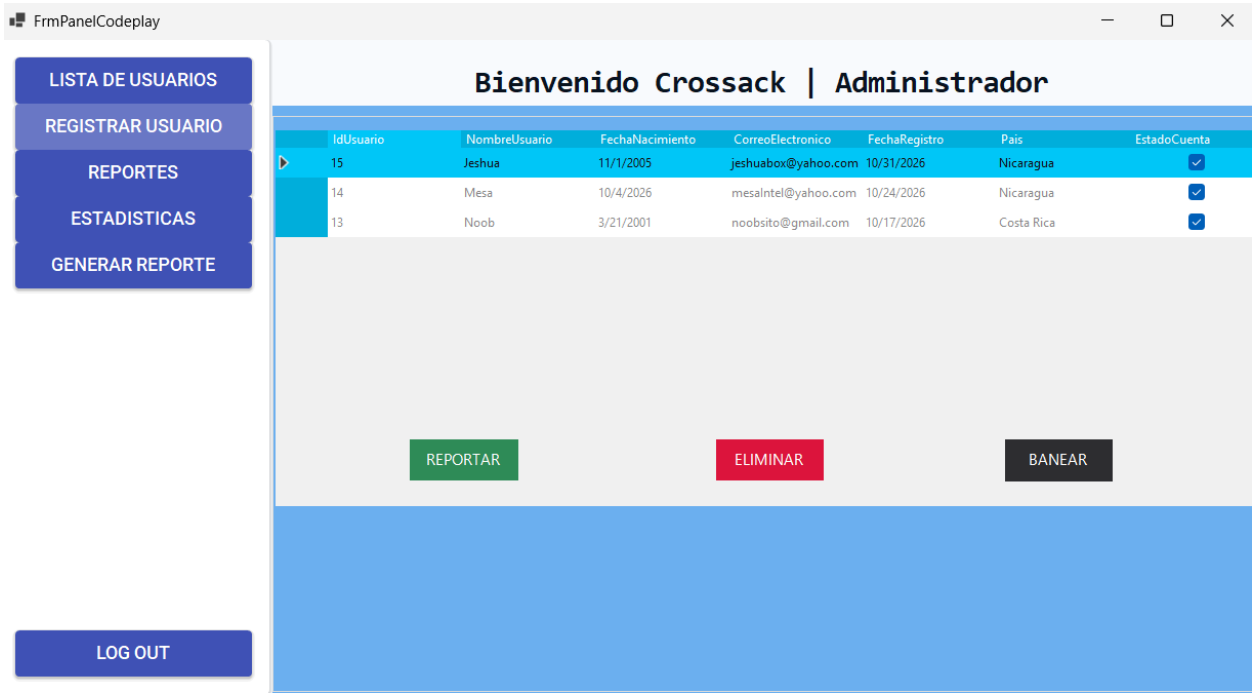


Figura 4. Si todas las acciones han sido ya realizadas dentro del panel de registro, haga click en el botón “Salir” para poder regresar al panel de administrador (esto solo funciona si ingresa con credenciales de administrador, si hace esto como operador, al hacer click en el botón “Salir” se regresará de nuevo a la ventana de inicio de sesión).

Figura 5. El programa regresa nuevamente al panel de administrador.



Casos de uso

Para esta demostración, primeramente ingresaremos como operador, ya que son los encargados de agregar a los usuarios. Para esto, usaremos una de las credenciales de prueba previamente vista en los capítulos anteriores.

Figura 4.



Figura 5. Iniciamos sesión con las credenciales del usuario SoteloMaykel

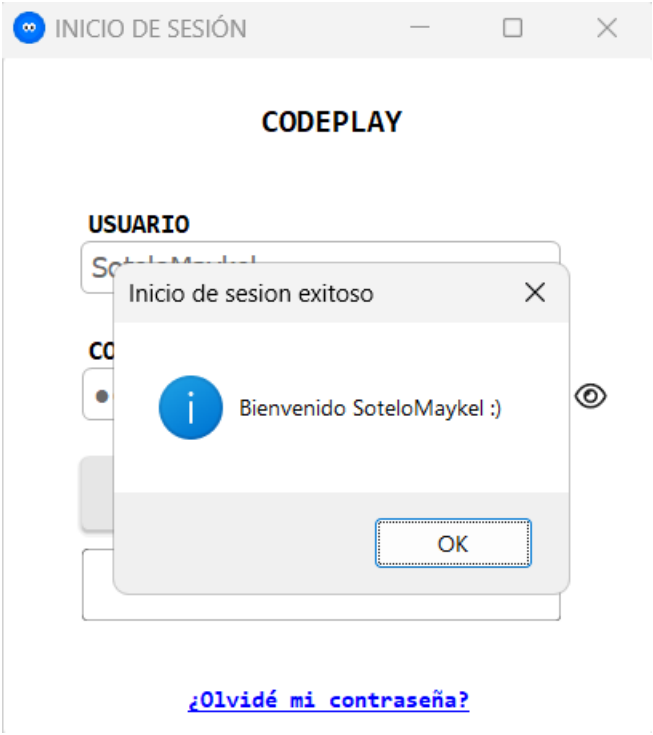


Figura 6. Procedemos a rellenar los campos con la nueva información del usuario y damos click en “Registrar”.



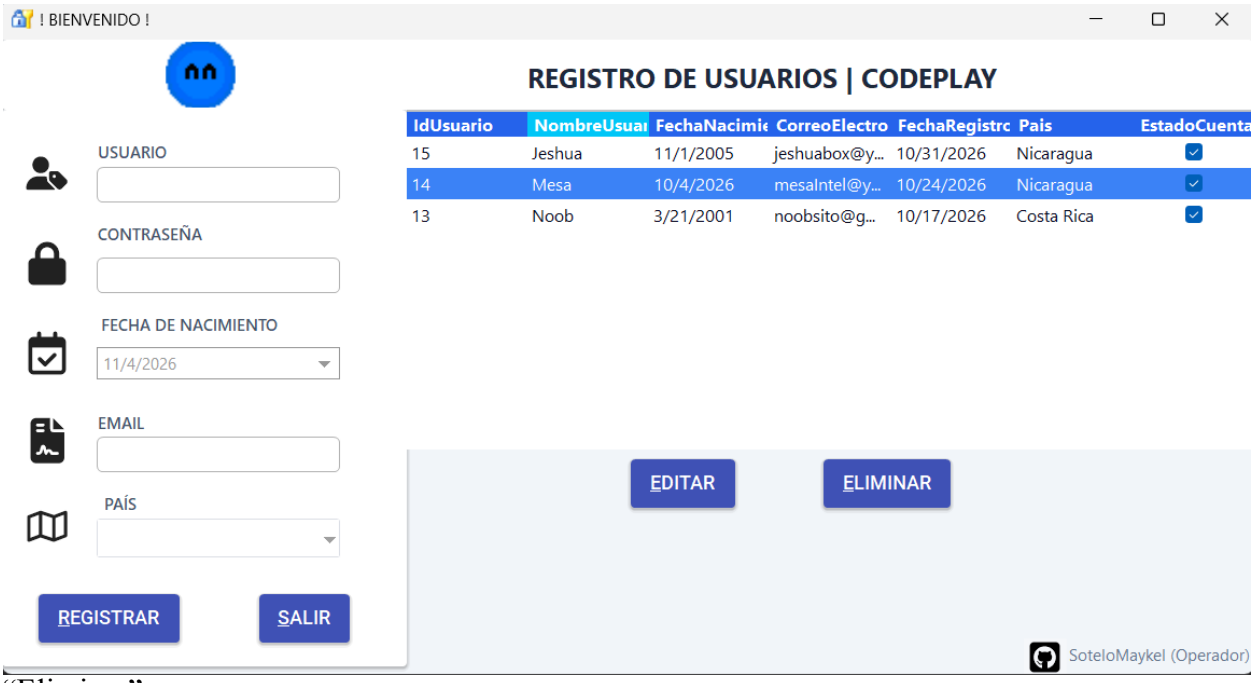
Figura 7. Una vez registramos la información, un mensaje de confirmación aparecerá si el proceso fue realizado con éxito.



Figura 8. Se observa que el nuevo usuario añadido podrá ser visualizado en el panel de usuarios.



Figura 9. Si deseamos eliminar un usuario ya registrado, primero seleccionamos el usuario haciendo clic sobre el (para este ejemplo usaremos a “Mesa”), luego presionar el botón



“Eliminar”

Figura 10. Un mensaje aparecerá indicándonos si estamos seguros de eliminar el usuario seleccionado, confirmamos la acción dando clic en “Yes”

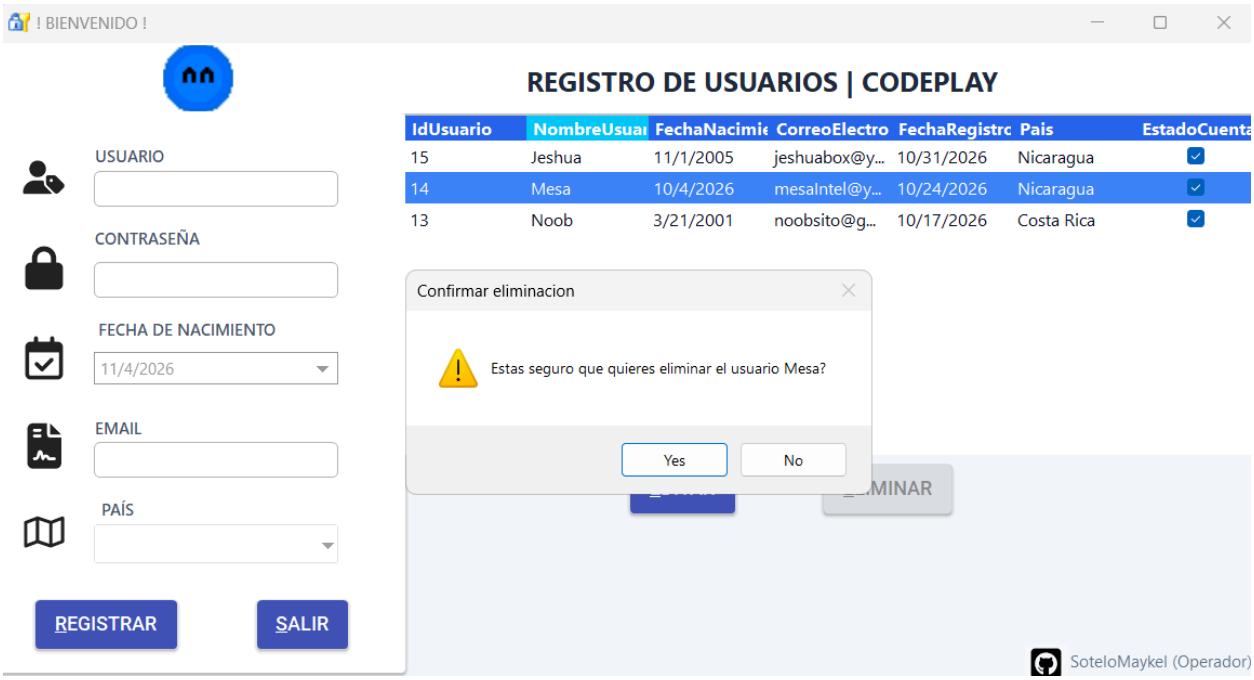


Figura 11. Al hacer clic el usuario habrá sido eliminado del registro.

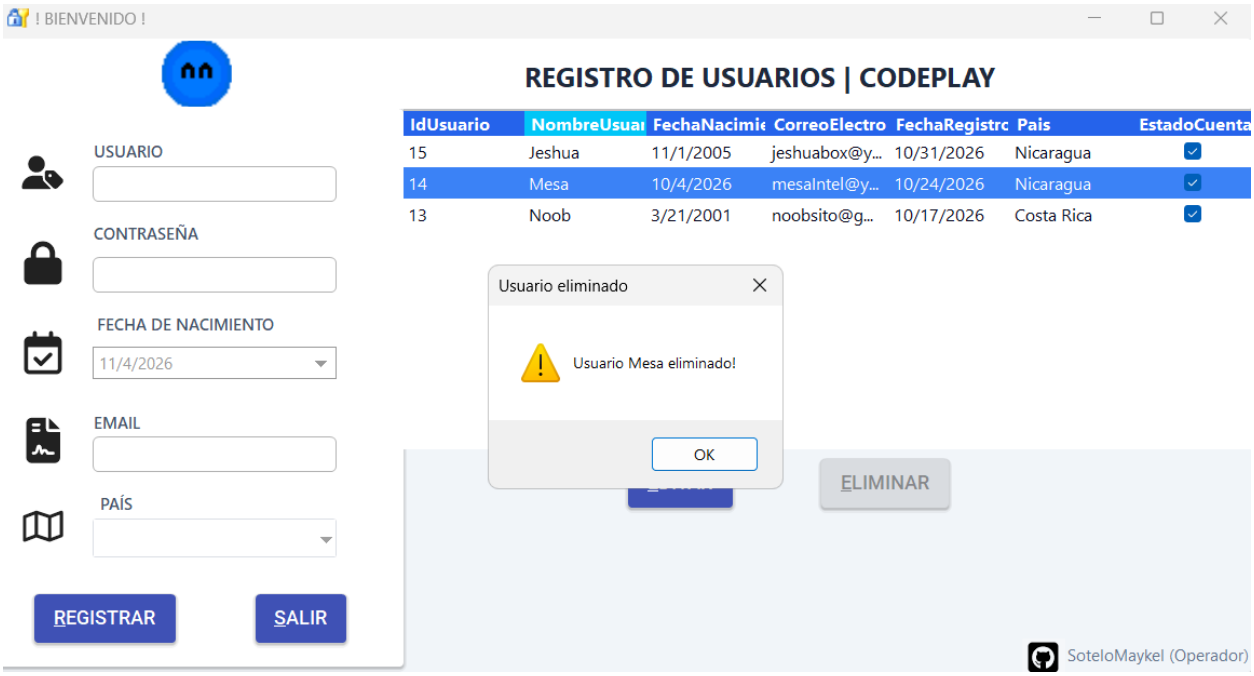


Figura 12. Usuario Eliminado



Manejo de errores comunes

En este apartado se abordarán los principales manejos de excepciones que el programa efectúa si el usuario o cualquier acción no sigue el protocolo establecido, estos utilizan las funciones Try-Catch para capturar cualquiera de estas excepciones.

Figura 1. Si alguno de los operadores o administradores olvida su contraseña o credenciales de acceso, tendrá la opción de cambiar la contraseña usando la opción “Olvide Mi contraseña”



Figura 2. Para poder reestablecer una contraseña nueva, se tendrá que confirmar la identidad del usuario (si la persona no confirma la identidad de su usuario, no podrá reestablecer la contraseña, esta información es obligatoria).

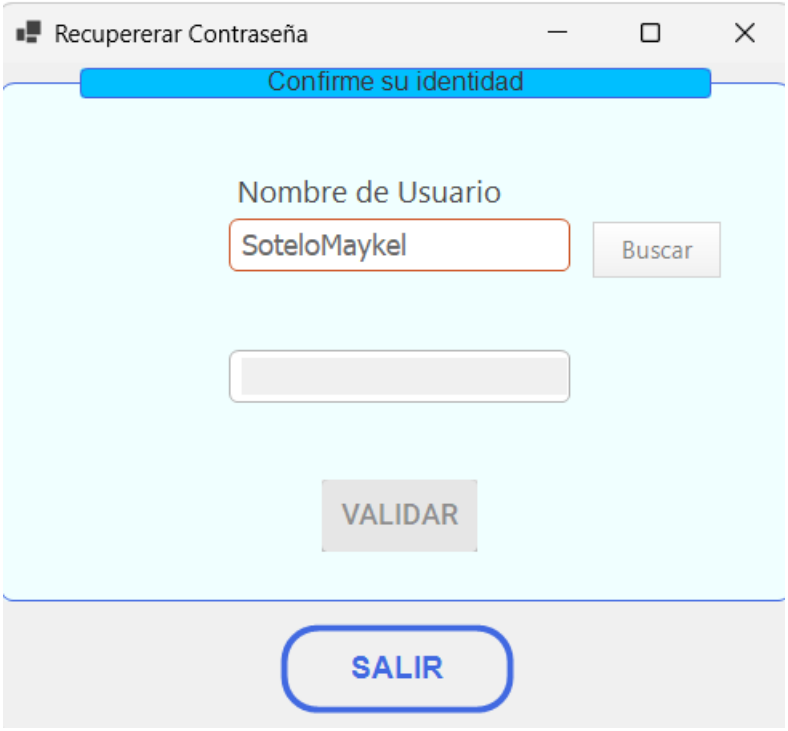


Figura 3. Si el usuario es el correcto, haga clic en buscar y tendrá que responder una pregunta secreta para poder validar su usuario poder reestablecer su contraseña (esta pregunta es previamente establecida por el equipo a cargo de la programación de credenciales).

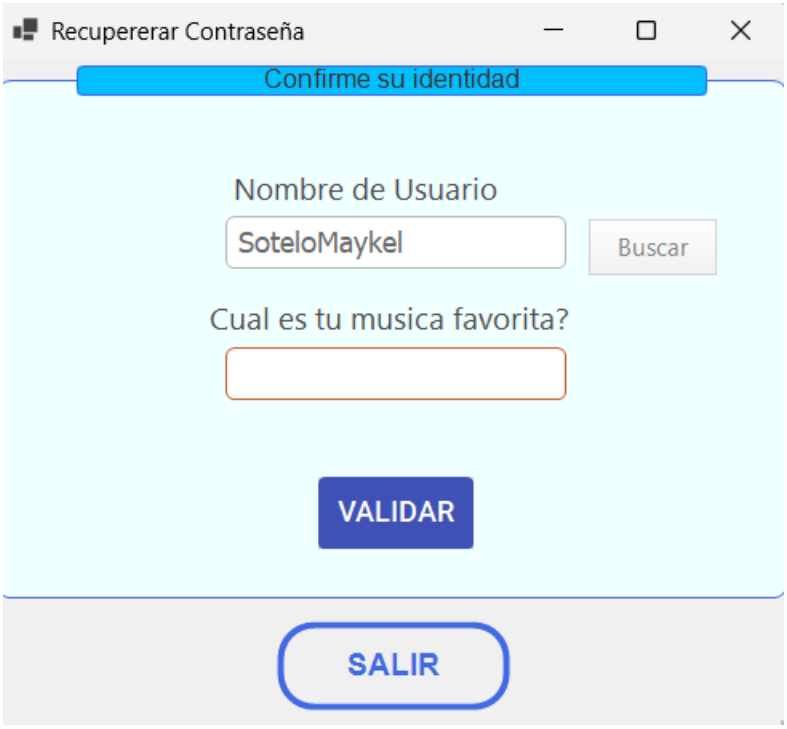


Figura 4. Si la pregunta fue contestada correctamente, el programa le mostrará la siguiente ventana para reestablecer la contraseña, introduzca la nueva contraseña y haga clic en “Cambiar”.



Glosario

- **Back-Office (Administración Interna):** Parte del sistema de software a la que solo tienen acceso los administradores y operadores de soporte para gestionar usuarios, auditar reportes y registrar ventas, sin estar expuesta al público general.
- **Baneo (*Ban*):** Sanción disciplinaria permanente aplicada a la cuenta de un jugador por infringir las normas de la comunidad. Inhabilita el acceso del usuario (EstadoCuenta = Inactivo/Baneado), pero conserva su registro histórico en la base de datos por integridad contable y de auditoría.
- **Estado de Cuenta (EstadoCuenta):** Indicador booleano o categórico dentro de la tabla de usuarios que refleja si una cuenta se encuentra activa y habilitada para operar en el videojuego o si ha sido restringida.
- **Moderación de Comunidad:** Proceso de supervisión, evaluación y sanción del comportamiento de los jugadores dentro de la plataforma, ejecutado mediante los flujos de *Reportar*, *Suspender*, *Banear* o *Eliminar*.
- **Puntos de Dolor (*Pain Points*):** Problemas operativos específicos que experimentan los actores del negocio antes de la automatización, tales como la fragmentación de datos, lentitud en la generación de reportes o errores manuales de registro.
- **Reporte en Revisión:** Estado transitorio en el que se encuentra una incidencia levantada contra un jugador mientras el administrador evalúa las pruebas antes de emitir una sanción definitiva.

- **Rol de Usuario (RolUsuario):** Nivel de privilegio asignado a una cuenta dentro de Codeplay (por ejemplo, *Administrador* u *Operador*), el cual determina a qué formularios y acciones tiene acceso tras iniciar sesión.
- **Suspensión Temporal:** Restricción transitoria de acceso aplicada a un usuario reportado, manteniendo su cuenta bloqueada por un periodo determinado sin llegar al baneo definitivo.
- **Trazabilidad:** Capacidad del sistema para rastrear el historial completo de un usuario desde su fecha de registro (FechaRegistro), pasando por sus compras de videojuegos, hasta sus incidencias disciplinarias.
- **Arquitectura Cliente-Servidor:** Modelo de diseño de software en el que la aplicación de escritorio (cliente en Windows Forms) procesa la interfaz y la lógica de presentación, mientras que las peticiones de datos se centralizan en un servidor de base de datos (**Microsoft SQL Server**).
- **ClosedXML:** Librería de código abierto para .NET que permite crear, leer y manipular archivos de hojas de cálculo de Microsoft Excel (.xlsx) de forma programática sin requerir que Excel esté instalado en el equipo, utilizada para el módulo de exportación de reportes.
- **Control de Acceso Basado en Roles (RBAC):** Mecanismo de seguridad que restringe el acceso al sistema según el perfil del usuario autenticado en la sesión activa (SesionActiva).
- **Cuadrícula de Datos (DataGridView / PoisonDataGridView):** Componente visual tabular de Windows Forms y ReaLTaiizor empleado para renderizar filas y columnas provenientes de consultas a la base de datos en modo de lectura estructurada.
- **Integridad Referencial:** Propiedad de las bases de datos relacionales (SQL Server) que garantiza que las relaciones entre tablas (por ejemplo, entre un *Usuario* y su historial de *Ventas* o *Reportes*) se mantengan consistentes, impidiendo registros huérfanos o eliminaciones accidentales.
- **Latencia Cero (Procesamiento Local):** Respuesta inmediata de la interfaz gráfica al filtrar, ordenar o conmutar paneles en memoria RAM dentro del entorno nativo de Windows, sin depender de tiempos de carga de red de un navegador web.
- **LINQ (*Language Integrated Query*):** Tecnología de Microsoft .NET que permite realizar consultas, filtrados y agrupaciones de colecciones de datos directamente desde el código en C# con sintaxis tipada.
- **NuGet:** Gestor de paquetes oficial para la plataforma .NET que permite integrar librerías de terceros en Visual Studio, tales como ReaLTaiizor, FontAwesome.Sharp o ClosedXML.
- **ReaLTaiizor:** Biblioteca de controles de interfaz de usuario (*UI*) de código abierto para Windows Forms que proporciona componentes basados en *Material Design* (MaterialCard, MaterialButton) y estilos personalizados (PoisonDataGridView, PoisonLabel).
- **Red de Área Local (LAN):** Red informática interna de alcance local en la que operan las estaciones de trabajo administrativas de Codeplay, manteniendo la base de datos aislada de ataques externos provenientes de Internet.
- **Sesión Activa (SesionActiva):** Clase o estructura estática en memoria encargada de almacenar temporalmente los datos del usuario autenticado (IdUsuario, NombreUsuario, RolUsuario) mientras la aplicación permanece abierta.
- **Superficie de Ataque:** Conjunto de todos los puntos vulnerables potenciales a través de los cuales un usuario no autorizado podría intentar ingresar o extraer datos de un sistema; en aplicaciones de escritorio internas, esta superficie es significativamente menor que en aplicaciones web públicas.
- **T-SQL (*Transact-SQL*):** Extensión propietaria del lenguaje SQL utilizada por Microsoft SQL Server para escribir consultas complejas, procedimientos almacenados, desencadenadores (*triggers*) y transacciones.

- **Windows Forms (WinForms):** Marco de trabajo (*framework*) de interfaz gráfica de usuario incluido en .NET para el desarrollo de aplicaciones nativas de escritorio sobre el sistema operativo Windows.

FAQ´s

1. **¿Por qué al iniciar sesión no veo el panel de administración ni las opciones de moderación?**
   - El sistema Codeplay cuenta con control de acceso basado en roles. Si tu cuenta tiene asignado el rol de usuario/operador estándar, serás dirigido automáticamente al módulo de registros (frmRegistros). Solo las cuentas con privilegios de Administrador tienen acceso al panel maestro (FrmPanelCodeplay).
2. **¿Qué debo hacer si olvidé mi contraseña de acceso?**
   - En la pantalla de inicio de sesión, haz clic en la opción de recuperación de contraseña o contacta directamente a un administrador del sistema para que restablezca tus credenciales desde la base de datos.
3. **¿Cómo cierro mi sesión de forma segura para que otro operador use el equipo?**
   - Utiliza siempre el botón **LOG OUT** ubicado en la esquina inferior izquierda del menú lateral. Esto limpiará los datos de la sesión activa en memoria y te devolverá a la ventana de autenticación. (Si entras como operador, usa siempre el botón **SALIR**, que te llevará de vuelta a la ventana de inicio de sesión).
4. **¿Cuál es la diferencia entre Reportar, Suspender, Banear y Eliminar a un jugador?**
   - **Reportar:** Marca la cuenta del jugador y la envía a la bandeja de *Usuarios Reportados* / *Reportes en Revisión* para que su comportamiento sea auditado sin bloquear su acceso todavía.
   - **Suspender:** Deshabilita temporalmente el acceso del jugador durante un periodo determinado.

- **Banear:** Bloquea de forma permanente el acceso de la cuenta (EstadoCuenta = Inactivo/Baneado), pero conserva su registro en la base de datos para mantener la integridad del historial de ventas y evitar que vuelva a registrarse con el mismo correo.
- **Eliminar:** Da de baja definitiva el registro del usuario en el sistema (sujeto a que no afecte historiales contables críticos).

5. ¿Cómo aplico una sanción a un usuario desde el panel?

- Ve a la sección Lista de Usuarios, selecciona la fila correspondiente al jugador en la tabla de datos y haz clic en uno de los botones inferiores (Reportar, Eliminar o Banear).

6. ¿Dónde puedo consultar los usuarios que tienen incidencias abiertas?

- Al hacer clic en el botón Reportes del menú lateral, se desplegará un submenú con tres filtros específicos: *Usuarios Reportados*, *Usuarios Suspendidos* y *Reportes en Revisión*.

7. ¿Cómo registro un nuevo usuario o una nueva venta en el sistema?

- Desde el menú lateral izquierdo, presiona el botón Registrar Usuario. Se abrirá el formulario de registro donde podrás ingresar los datos demográficos del cliente (nombre, fecha de nacimiento, correo electrónico, país) y asociar su transacción.

8. ¿En qué formato se exportan los informes al usar "Generar Reporte"?

- El botón Generar Reporte consolida los datos actuales de la vista o de las transacciones en un documento estructurado (como hojas de cálculo de Excel .xlsx), listo para ser almacenado en el equipo local y presentado a gerencia.

9. ¿El sistema necesita conexión a Internet para funcionar?

- Al ser una aplicación nativa de escritorio en Windows Forms, Codeplay solo requiere conexión a la red local (LAN) donde se encuentre alojado el servidor de base de datos Microsoft SQL Server.

10. ¿Por qué no puedo editar los datos de un jugador haciendo doble clic directamente sobre las celdas de la tabla?

- Por seguridad e integridad referencial, la cuadrícula de datos (dgvListaUsuarios) está configurada en modo de solo lectura (ReadOnly = true). Cualquier cambio de estado o modificación debe realizarse exclusivamente mediante los botones de acción autorizados.

11. ¿Qué hago si la tabla de usuarios aparece vacía al abrir el panel?

- Verifica que el servicio de SQL Server esté en ejecución en tu equipo o servidor local y presiona nuevamente el botón Lista de Usuarios para refrescar la consulta de datos.